LAPORAN PRAKTIKUM MODUL-3
FUNCTIONS & ARRAY

OLEH :

NAMA : MUHAMMAD IHSAN

NIM : 2025573010053

KELAS : TI – 1C

PRODI TEKNIK INFORMATIKA

POLITEKNIK NEGERI LHOKSEUMAWE

TAHUN 2025/2026

LEMBAR PENGESAHAN

Nama	: Muhammad Ihsan
NIM	: 2025573010053
Kelas	: TI -1C
Jurusan	: Teknologi Informasi & Komputer
Prodi	: Teknik Informatika
Hari / Jam	: Rabu 8 April 2026 / 07.30 – 11.25
Dosen pengampu	: Muhammad Reza Zulman S.ST.,M.Sc

Disetujui,

Dosen Pengampu

Muhammad Reza Zulman S.ST.,M.Sc

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum ini dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas pelaksanaan praktikum pada mata kuliah Struktur Data dan Algoritma.

Laporan ini membahas mengenai materi Function dan Array dalam bahasa pemrograman JavaScript, yang meliputi konsep dasar function, parameter dan return value, arrow function, callback, array, method array, hingga rekursi. Seluruh materi yang disajikan dalam laporan ini disusun berdasarkan modul praktikum yang telah diberikan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang.

Penulis juga mengucapkan terima kasih kepada dosen pengampu mata kuliah Struktur Data dan Algoritma serta semua pihak yang telah membantu dalam proses pelaksanaan praktikum dan penyusunan laporan ini.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep Function dan Array dalam pemrograman JavaScript.

Buket Rata, 8 April 2026

Muhammad Ihsan

DAFTAR ISI

LEMBAR PENGESAHAN 2
KATA PENGANTAR 3
DAFTAR ISI 4
BAB I 6
PENDAHULUAN 6
1.1 Latar Belakang 6
1.2 Tujuan Praktikum 6
BAB II 7
DASAR TEORI 7
2.1 Pengertian Function 7
2.2 Arrow Function 7
2.3 Parameter dan Return Value 8
2.4 Konsep Callback 8
2.5 Pengertian Array 9
2.6 Operasi Dasar pada Array 9
2.7 Method Iterasi Array 9
2.8 Konsep Rekursi 10
2.9 Hubungan Function dan Array dalam Pemrograman 11
BAB III 12
METODOLOGI PRAKTIKUM 12
3.1 Waktu dan Tempat 12
3.2 Alat dan Bahan 12
3.3 Langkah Kerja 12
BAB IV 29
HASIL DAN KESIMPULAN 29
4.1 Hasil Praktikum 29
4.2 Pembahasan 29
4.2.1 Function (Fungsi) 29
4.2.2 Array 30
4.2.3 Penggunaan Reduce untuk Statistik 30

4.2.4 Function Rekursif 30
4.2.5 Analisis Hasil 30
BAB V 31
PENUTUP 31
5.1 Kesimpulan 31
5.2 Saran 31

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman JavaScript, penggunaan function dan array merupakan dasar yang sangat penting dalam pengolahan data dan pembuatan program yang efisien. Function digunakan untuk mengelompokkan kode agar dapat digunakan kembali, sedangkan array digunakan untuk menyimpan banyak data dalam satu variabel.

Pada praktikum ini, mahasiswa mempelajari konsep function, arrow function, callback, array, serta berbagai method array seperti forEach, map, filter, dan reduce. Selain itu, juga diperkenalkan konsep rekursi sebagai teknik lanjutan dalam pemrograman.

1.2 Tujuan Praktikum

Berdasarkan modul, tujuan praktikum ini adalah:

- Memahami konsep function beserta parameter dan return value
- Memahami scope variabel
- Menggunakan arrow function
- Memahami callback dalam JavaScript
- Mengelola array dan method array
- Menggunakan iterasi array (forEach, map, filter, reduce)
- Memahami konsep rekursi

BAB II

DASAR TEORI

2.1 Pengertian Function

Function merupakan sekumpulan instruksi atau perintah dalam program yang dirancang untuk menjalankan tugas tertentu dan dapat dipanggil kembali kapan saja sesuai kebutuhan. Dengan adanya function, penulisan program menjadi lebih terstruktur, efisien, dan mudah dipelihara karena tidak perlu mengulang kode yang sama berkali-kali.

Dalam pemrograman, function memiliki beberapa komponen penting, yaitu parameter sebagai nilai masukan, proses atau isi function, serta return value sebagai hasil keluaran. Selain itu, function juga berkaitan erat dengan konsep scope, yaitu ruang lingkup variabel. Variabel yang dideklarasikan di dalam function hanya dapat digunakan di dalam function tersebut (local scope), sedangkan variabel yang berada di luar function disebut global scope dan dapat diakses secara lebih luas.

Penggunaan function sangat penting dalam penerapan prinsip efisiensi dan modularitas program**,** sehingga program dapat dikembangkan dengan lebih mudah dan sistematis.

2.2 Arrow Function

Arrow function merupakan bentuk penulisan function yang lebih modern dan ringkas dalam JavaScript. Konsep ini diperkenalkan untuk mempermudah penulisan function terutama pada operasi yang sederhana dan tidak membutuhkan struktur kompleks.

Arrow function memiliki kelebihan dalam hal sintaks yang lebih singkat dan mudah dibaca. Selain itu, arrow function sering digunakan dalam pemrograman modern karena lebih praktis, terutama saat digunakan bersama dengan method array atau callback.

Penggunaan arrow function membantu programmer dalam menulis kode yang lebih sederhana tanpa mengurangi fungsi utama dari program itu sendiri.

2.3 Parameter dan Return Value

Parameter adalah nilai yang diberikan ke dalam function sebagai input untuk diproses. Parameter memungkinkan function menjadi lebih fleksibel karena dapat menerima berbagai nilai yang berbeda saat dipanggil.

Return value adalah nilai yang dihasilkan oleh function setelah proses selesai. Nilai ini dapat digunakan kembali dalam program, baik untuk ditampilkan maupun untuk diproses lebih lanjut.

Dalam JavaScript, terdapat juga konsep **default parameter**, yaitu nilai awal yang digunakan apabila parameter tidak diberikan saat function dipanggil. Hal ini membuat function tetap dapat berjalan meskipun tidak semua input diberikan.

Pemahaman terhadap parameter dan return value sangat penting karena menjadi dasar dalam membangun function yang dinamis dan dapat digunakan dalam berbagai kondisi.

2.4 Konsep Callback

Callback adalah function yang dikirim sebagai argumen ke dalam function lain untuk kemudian dipanggil pada waktu tertentu. Konsep ini merupakan salah satu dasar penting dalam JavaScript, terutama dalam pengelolaan proses yang tidak berjalan secara berurutan (asynchronous).

Callback memungkinkan sebuah function untuk menjadi lebih fleksibel karena dapat menerima fungsi lain sebagai parameter. Dengan demikian, suatu proses dapat menghasilkan output yang berbeda tergantung pada callback yang diberikan.

Konsep callback banyak digunakan dalam berbagai fitur JavaScript seperti event handling, timer, serta dalam method array. Dengan memahami callback, programmer dapat mengontrol alur eksekusi program dengan lebih baik.

2.5 Pengertian Array

Array adalah struktur data yang digunakan untuk menyimpan sekumpulan nilai dalam satu variabel secara terurut dan memiliki indeks. Indeks pada array dimulai dari angka 0, sehingga elemen pertama berada pada indeks ke-0.

Array bersifat dinamis, yang berarti jumlah elemen di dalamnya dapat bertambah atau berkurang sesuai kebutuhan. Selain itu, array dalam JavaScript dapat menyimpan berbagai jenis data dalam satu tempat, meskipun dalam praktiknya lebih baik menggunakan tipe data yang seragam untuk menjaga konsistensi.

Penggunaan array sangat membantu dalam pengelolaan data dalam jumlah banyak, sehingga mempermudah proses pengolahan dan manipulasi data dalam program.

2.6 Operasi Dasar pada Array

Array menyediakan berbagai operasi dasar yang dapat digunakan untuk memanipulasi data. Operasi tersebut meliputi penambahan data, penghapusan data, pencarian data, serta pengambilan sebagian data dari array.

Dengan adanya operasi-operasi tersebut, pengguna dapat dengan mudah mengatur isi array sesuai kebutuhan. Misalnya, menambahkan elemen baru ke dalam array, menghapus elemen tertentu, atau mencari posisi suatu elemen.

Operasi dasar ini merupakan fondasi penting dalam pengolahan data menggunakan array.

2.7 Method Iterasi Array

Method iterasi array adalah metode yang digunakan untuk mengolah setiap elemen dalam array secara otomatis dengan bantuan function. Metode ini bekerja dengan cara menjalankan sebuah function pada setiap elemen array.

Beberapa method iterasi yang umum digunakan antara lain:

- **forEach**, yang digunakan untuk menjalankan proses pada setiap elemen tanpa menghasilkan nilai baru
- **map**, yang digunakan untuk menghasilkan array baru dari hasil transformasi elemen
- **filter**, yang digunakan untuk menyaring elemen berdasarkan kondisi tertentu
- **reduce**, yang digunakan untuk menggabungkan seluruh elemen menjadi satu nilai

Penggunaan method iterasi ini membuat kode menjadi lebih ringkas, mudah dibaca, dan efisien dibandingkan dengan perulangan biasa.

2.8 Konsep Rekursi

Rekursi adalah teknik pemrograman di mana sebuah function memanggil dirinya sendiri untuk menyelesaikan suatu masalah secara bertahap. Teknik ini digunakan untuk memecah masalah besar menjadi bagian yang lebih kecil hingga mencapai kondisi tertentu.

Dalam rekursi terdapat dua komponen utama, yaitu:

- **Base case**, yaitu kondisi berhenti agar function tidak berjalan terus-menerus
- **Recursive case**, yaitu proses pemanggilan function itu sendiri

Rekursi sering digunakan dalam berbagai algoritma, seperti perhitungan matematika, pencarian data, serta struktur data seperti tree dan graph. Namun, penggunaannya harus diperhatikan dengan baik agar tidak menyebabkan kesalahan seperti pemanggilan tanpa henti.

2.9 Hubungan Function dan Array dalam Pemrograman

Function dan array merupakan dua konsep yang saling berkaitan dalam pemrograman. Function digunakan untuk mengolah data, sedangkan array digunakan untuk menyimpan data tersebut.

Dengan menggabungkan keduanya, programmer dapat membuat program yang lebih fleksibel dan efisien, seperti melakukan perhitungan, penyaringan data, maupun transformasi data dalam jumlah besar.

Penggunaan function bersama array, terutama dengan method iterasi, sangat membantu dalam menyelesaikan berbagai permasalahan dalam pemrograman secara efektif dan terstruktur.

BAB III

METODOLOGI PRAKTIKUM

3.1 Waktu dan Tempat

- Waktu: 8 April 2026
- Tempat: Laboratorium komputer 106

3.2 Alat dan Bahan

- Laptop/PC
- Visual Studio Code
- Node.js
- Terminal / Git Bash

3.3 Langkah Kerja

Langkah 1 || Masuk ke repository dengan memasukan perintah `cd nama repository`

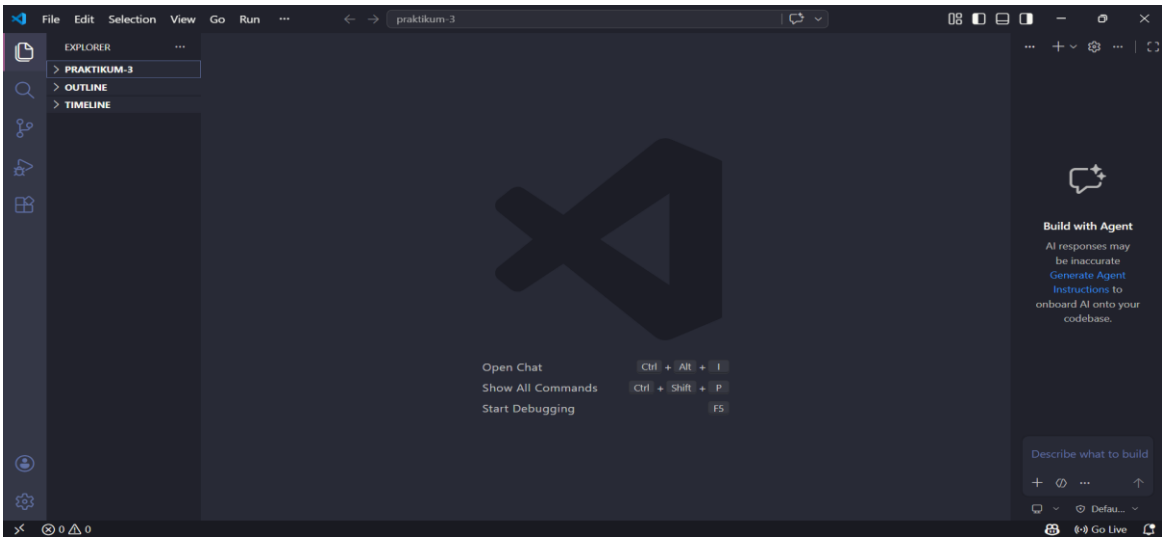
```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~
$ cd 2025573010053_Algoritma_Dan_Struktur_Data
```

Langkah 2 || Membuat folder dengan masukan perintah `mkdir praktikum-3` setelah itu masuk ke folder praktikum-3 dengan memasukan perintah `cd praktikum-3`

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010053_Algoritma_Dan_Struktur_Dat
a (main)
$ cd praktikum-3
```

Langkah 3 || Inisialisasi project dengan memasukan perintah `npm init -y`

Langkah 4 || Masuk ke visual studio code dengan memasukan perintah `code` .



Langkah 5 || Membuat file dengan cara masuk ke terminal masukan perintah `touch 01-function-dasar.js`

JS 01-function-dasar.js

```js
function kuadratBiasa(x) {
  return x * x;
}

// cara baru (arrow function - ekuivalen)
const kuadrat = (x) => {
  return x * x;
};

// Arrow functio ringkas (implicit return)
const kuadratRingkas = (x) => x * x;

console.log("=== Perbandingan Penulisan ===");
console.log("Biasa   :", kuadratBiasa(5)); //25
console.log("Arrow   :", kuadrat(5)); //25
console.log("Ringkas   :", kuadratRingkas(5)); //25

// --- 2. Arrow function dengan beberapa parameter ---
const luas = (panjang, lebar) => panjang * lebar;
const salam = (nama, waktu) => "selamat ${waktu}, ${nama}!";

```

```javascript
console.log("Luas 4x6 :", luas(4, 6));
console.log(salam("Budi", "pagi"));

//  --- 3. Callback: function sebagai argumen ---
function lakukanOperasi(angka, operasiCallback) {
  console.log("Angka awal: ${angka}");
  const hasil = operasiCallback(angka);
  console.log("Hasil setelah operasi: ${hasil}");
}

console.log("\n=== Callback ==="); //kirim function sebagai argumen
lakukanOperasi(7, kuadratRingkas); //kirim arrow function langsung
lakukanOperasi(10, (x) => x + 100); //kirim function biasa langsung
lakukanOperasi(20, function (x) {
  return x / 2;
});

// --- 4. setTimeout sebagai contoh callback nyata ---
console.log("\n=== setTimeout (Callback)===");
console.log("pesan 1: Sebelum timer");

setTimeout(() => {
  console.log("pesan 3: Ini dari dalam setTimeout (setelah 1 detik)");
}, 1000); //1000 milidetik = 1 detik

console.log("pesan 2: Setelah mendaftarkan timer");

```

Outputnya :

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010053_Algoritma_Dan_Struktur_Data/praktikum-3 (main)
$ node 01-function-dasar.js
=== Perbandingan Penulisan ===
Biasa   : 25
Arrow   : 25
Ringkas   : 25
Luas 4x6 : 24
selamat ${waktu}, ${nama}!

=== Callback ===
Angka awal: ${angka}
Hasil setelah operasi: ${hasil}
Angka awal: ${angka}
Hasil setelah operasi: ${hasil}
Angka awal: ${angka}
Hasil setelah operasi: ${hasil}

=== setTimeout (Callback)===
pesan 1: Sebelum timer
pesan 2: Setelah mendaftarkan timer
pesan 3: Ini dari dalam setTimeout (setelah 1 detik)
```

Langkah 6 || Membuat file 02-arrow-callback

JS 02-arrow-callback.js

```javascript
function kuadratBiasa(x) {
  return x * x;
}
// Cara baru (arrow function - ekuivalen)
const kuadrat = (x) => {
  return x * x;
};
// Arrow function ringkas (implicit return)
const kuadratRingkas = (x) => x * x;
console.log("=== Perbandingan Penulisan ===");
console.log("Biasa :", kuadratBiasa(5)); // 25
console.log("Arrow :", kuadrat(5)); // 25
console.log("Ringkas :", kuadratRingkas(5)); // 25
// --- 2. Arrow function dengan beberapa parameter ---
const luas = (panjang, lebar) => panjang * lebar;
const salam = (nama, waktu) => `Selamat ${waktu}, ${nama}!`;
console.log("Luas 4x6 :", luas(4, 6));
console.log(salam("Budi", "Pagi"));
// --- 3. Callback: function sebagai argumen ---
function lakukanOperasi(angka, operasiCallback) {
  console.log(`Angka awal: ${angka}`);
  const hasil = operasiCallback(angka);
  console.log(`Hasil setelah operasi: ${hasil}`);
}
console.log("\n=== Callback ===");
lakukanOperasi(7, kuadratRingkas); // kirim function sebagai argumen
lakukanOperasi(10, (x) => x + 100); // kirim arrow function langsung
lakukanOperasi(20, function (x) {
  // kirim function biasa langsung
  return x / 2;
});
// --- 4. setTimeout sebagai contoh callback nyata ---
console.log("\n=== setTimeout (Callback) ===");
console.log("Pesan 1: Sebelum timer");
setTimeout(() => {
  console.log("Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)");
}, 1000); // 1000 milidetik = 1 detik
console.log("Pesan 2: Setelah mendaftarkan timer");
```

Outputnya :

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010053_Algoritma_Dan_Struktur_Data/praktikum-3 (main)
$ node 02-arrow-callback.js
=== Perbandingan Penulisan ===
Biasa : 25
Arrow : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout (Callback) ===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer
Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)
```

Langkah 7 || Membuat file 03-array-dasar.js

JS 03-array-dasar.js

```
const mahasiswa = ["Budi", "Siti", "Ahmad", "Rina"];
const nilai = [85, 92, 78, 95, 88];
console.log("=== Array Awal ===");
console.log("Mahasiswa:", mahasiswa);
console.log("Nilai :", nilai);
console.log("Jumlah mahasiswa:", mahasiswa.length);
// --- 2. Mengakses elemen ---
console.log("\n=== Akses Elemen ===");
console.log("Elemen pertama :", mahasiswa[0]); // indeks 0
console.log("Elemen ketiga :", mahasiswa[2]); // indeks 2
console.log("Elemen terakhir:", mahasiswa[mahasiswa.length - 1]);
// --- 3. Mengubah elemen ---
mahasiswa[1] = "Siti Rahayu"; // ubah elemen indeks 1
console.log("\nSetelah diubah:", mahasiswa);
// --- 4. Menambah dan menghapus elemen ---
console.log("\n=== Manipulasi Array ===");
```

```javascript
console.log("\n=== Manipulasi Array ===");
mahasiswa.push("Doni"); // tambah di akhir
console.log("Setelah push :", mahasiswa);
mahasiswa.unshift("Zahra"); // tambah di awal
console.log("Setelah unshift :", mahasiswa);
const dihapusAkhir = mahasiswa.pop(); // hapus dari akhir
console.log("Dihapus (pop) :", dihapusAkhir);
console.log("Setelah pop :", mahasiswa);
const dihapusAwal = mahasiswa.shift(); // hapus dari awal
console.log("Dihapus (shift) :", dihapusAwal);
console.log("Setelah shift :", mahasiswa);
// --- 5. Mencari elemen ---
console.log("\n=== Pencarian ===");
console.log("Indeks Ahmad :", mahasiswa.indexOf("Ahmad"));
console.log("Ada Rina? :", mahasiswa.includes("Rina"));
console.log("Ada Budi? :", mahasiswa.includes("Budi"));
// --- 6. Mengiris array dengan slice ---
const sebagian = nilai.slice(1, 4); // dari indeks 1 sampai sebelum 4
console.log("\nNilai asli :", nilai);
console.log("Slice [1,4] :", sebagian);
```

Outputnya :

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010053_Algoritma_Dan_Struktur_Data/praktikum-3 (main)
$ node 03-array-dasar.js
=== Array Awal ===
Mahasiswa: [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai : [ 85, 92, 78, 95, 88 ]
Jumlah mahasiswa: 4

=== Akses Elemen ===
Elemen pertama : Budi
Elemen ketiga : Ahmad
Elemen terakhir: Rina

Setelah diubah: [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Manipulasi Array ===
Setelah push : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Setelah unshift : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Dihapus (pop) : Doni
Setelah pop : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
Dihapus (shift) : Zahra
Setelah shift : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Pencarian ===
Indeks Ahmad : 2
Ada Rina? : true
Ada Budi? : true

Nilai asli : [ 85, 92, 78, 95, 88 ]
Slice [1,4] : [ 92, 78, 95 ]
```

Langkah 8 || Membuat file 04-array-method.js

JS 04-array-method.js

```javascript
const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];
// --- 1. forEach: menjalankan sesuatu untuk setiap elemen ---
console.log("=== forEach: Tampilkan Semua Nilai ===");
nilaiMahasiswa.forEach((nilai, indeks) => {
  console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
});
// --- 2. map: transformasi setiap elemen menjadi nilai baru ---
// Konversi nilai angka ke grade huruf
const gradeHuruf = nilaiMahasiswa.map((nilai) => {
  if (nilai >= 90) return "A";
  if (nilai >= 80) return "B";
  if (nilai >= 70) return "C";
  if (nilai >= 60) return "D";
  return "E";
});
console.log("\n=== map: Nilai ke Grade ===");
console.log("Nilai :", nilaiMahasiswa);
console.log("Grade :", gradeHuruf);
// --- 3. filter: saring elemen yang memenuhi kondisi ---
const nilaiLulus = nilaiMahasiswa.filter((nilai) => nilai >= 60);
const nilaiGagal = nilaiMahasiswa.filter((nilai) => nilai < 60);
console.log("\n=== filter: Lulus dan Tidak Lulus ===");
console.log("Semua nilai :", nilaiMahasiswa);
console.log("Nilai lulus :", nilaiLulus);
console.log("Nilai gagal :", nilaiGagal);
// --- 4. reduce: mereduksi array menjadi satu nilai ---
const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
  return akumulator + nilai;
}, 0); // 0 adalah nilai awal akumulator
const rataRata = totalNilai / nilaiMahasiswa.length;
console.log("\n=== reduce: Statistik Nilai ===");
console.log("Total nilai :", totalNilai);
console.log("Rata-rata :", rataRata.toFixed(2));
// --- 5. Menggabungkan beberapa method (method chaining) ---
const rataRataNilaiLulus = nilaiMahasiswa
  .filter((nilai) => nilai >= 60) // ambil yang lulus dulu
  .reduce((acc, val, idx, arr) => {
    return acc + val / arr.length; // bagi saat proses
  }, 0);
console.log("\n=== Method Chaining ===");
console.log("Rata-rata nilai lulus:", rataRataNilaiLulus.toFixed(2));
```

Outputnya :

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010053_Algoritma_Dan_Struktur_Data/praktikum-3 (main)
$ node 04-array-method.js
=== forEach: Tampilkan Semua Nilai ===
 Mahasiswa-1: 75
 Mahasiswa-2: 90
 Mahasiswa-3: 55
 Mahasiswa-4: 82
 Mahasiswa-5: 68
 Mahasiswa-6: 95
 Mahasiswa-7: 48
 Mahasiswa-8: 77

=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
  'E', 'C'
]

=== filter: Lulus dan Tidak Lulus ===
Semua nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Nilai lulus : [ 75, 90, 82, 68, 95, 77 ]
Nilai gagal : [ 55, 48 ]
```

Langkah 9 || Membuat file 05-rekursi.js

JS 05-rekursi.js

```javascript
function faktorial(n) {
  // Base case: berhenti saat n = 1
  if (n <= 1) return 1;
  // Recursive case: n × faktorial(n-1)
  return n * faktorial(n - 1);
}
console.log("=== Faktorial ===");
console.log("0! =", faktorial(0)); // 1 (konvensi matematika)
console.log("1! =", faktorial(1)); // 1
console.log("5! =", faktorial(5)); // 120
console.log("7! =", faktorial(7)); // 5040
```

```javascript
// --- 2. Fibonacci: F(n) = F(n-1) + F(n-2) ---
// Deret: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
function fibonacci(n) {
  // Base case
  if (n === 0) return 0;
  if (n === 1) return 1;
  // Recursive case
  return fibonacci(n - 1) + fibonacci(n - 2);
}
console.log("\n=== Fibonacci ===");
for (let i = 0; i <= 8; i++) {
  process.stdout.write(fibonacci(i) + " ");
}
console.log(""); // pindah baris
// --- 3. Rekursi untuk menelusuri array ---
function jumlahArray(arr, indeks = 0) {
  // Base case: sudah sampai akhir array
  if (indeks === arr.length) return 0;
  // Recursive case: elemen saat ini + jumlah sisa array
  return arr[indeks] + jumlahArray(arr, indeks + 1);
}
const angka = [3, 7, 2, 9, 5];
console.log("\n=== Jumlah Array dengan Rekursi ===");
console.log("Array :", angka);
console.log("Jumlah :", jumlahArray(angka)); // 26
// --- 4. Countdown rekursif ---
function countdown(n) {
  if (n < 0) {
    console.log("Selesai!");
    return;
  }
  console.log(n);
  countdown(n - 1);
}
console.log("\n=== Countdown Rekursif ===");
countdown(5);
```

Outputnya :

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010053_Algoritma_Dan_Struktur_Data/praktikum-3 (main)
$ node 05-rekursi.js
=== Faktorial ===
0! = 1
1! = 1
5! = 120
7! = 5040

=== Fibonacci ===
0 1 1 2 3 5 8 13 21

=== Jumlah Array dengan Rekursi ===
Array : [ 3, 7, 2, 9, 5 ]
Jumlah : 26

=== Countdown Rekursif ===
5
4
3
2
1
0
Selesai!
```

Langkah 10 || Mengerjakan latihan yang sudah ada di modul,latihan ditambahkan pada file yang sudah kita buat sebelumnya

Latihan 1

```javascript
// Function penjumlahan
function tambah(a, b) {
  return a + b;
}

// Function pengurangan
function kurang(a, b) {
  return a - b;
}

// Function perkalian
function kali(a, b) {
  return a * b;
}

// Function pembagian
function bagi(a, b) {
  if (b === 0) {
    console.log("Error: tidak bisa membagi dengan nol");
    return null;
  }
  return a / b;
}

// Function kalkulator utama
function kalkulator(a, operator, b) {
  switch (operator) {
    case "+":
      return tambah(a, b);
    case "-":
      return kurang(a, b);
    case "*":
      return kali(a, b);
    case "/":
      return bagi(a, b);
    default:
      console.log("Operator tidak dikenali");
      return null;
  }
}

// Pengujian (minimal 5)
console.log(kalkulator(10, "+", 5)); // 15
console.log(kalkulator(10, "-", 5)); // 5
console.log(kalkulator(10, "*", 5)); // 50
console.log(kalkulator(10, "/", 5)); // 2
console.log(kalkulator(10, "/", 0)); // Error + null
```

Outputnya :

```
15
5
50
2
Error: tidak bisa membagi dengan nol
null
pesan 3: Ini dari dalam setTimeout (setelah 1 detik)
```

Latihan 2

```javascript
// Arrow function huruf besar
const keHurufBesar = (teks) => {
  return teks.toUpperCase();
};

// Arrow function tambah tanda seru
const tambahSeru = (teks) => {
  return teks + "!!!";
};

// Arrow function hitung jumlah kata
const hitungKata = (teks) => {
  return teks.split(" ").length;
};

// Function dengan callback
function prosesKalimat(kalimat, transformasiCallback) {
  const hasil = transformasiCallback(kalimat);
  console.log(hasil);
}

// Kalimat uji
const kalimat = "belajar javascript itu menyenangkan";
// Pengujian
prosesKalimat(kalimat, keHurufBesar);
// Output: BELAJAR JAVASCRIPT ITU MENYENANGKAN

prosesKalimat(kalimat, tambahSeru);
// Output: belajar javascript itu menyenangkan!!!

prosesKalimat(kalimat, hitungKata);
// Output: 4

```

Outputnya :

```
=== setTimeout (Callback) ===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer
BELAJAR JAVASCRIPT ITU MENYENANGKAN
belajar javascript itu menyenangkan!!!
4
Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)
```

Latihan 3

```javascript
// 1. Membuat array daftar belanja
let daftarBelanja = ["Beras", "Gula", "Minyak", "Telur", "Garam"];

// 2. Menampilkan isi daftar dengan nomor urut (for loop)
console.log("Daftar Belanja:");
for (let i = 0; i < daftarBelanja.length; i++) {
  console.log(i + 1 + ". " + daftarBelanja[i]);
}

// 3. Menambahkan 2 item baru
daftarBelanja.push("Susu");
daftarBelanja.push("Kopi");

console.log("\nSetelah ditambah:");
console.log(daftarBelanja);

// 4. Menghapus item pertama
let itemDihapus = daftarBelanja.shift();
console.log("\nItem yang dihapus:", itemDihapus);

// 5. Cek apakah 'Susu' ada di daftar
let cekSusu = daftarBelanja.includes("Susu");
if (cekSusu) {
  console.log("\n'Susu' ada di dalam daftar belanja");
} else {
  console.log("\n'Susu' tidak ada di dalam daftar belanja");
}

// 6. Menampilkan jumlah item akhir
console.log("\nJumlah item akhir:", daftarBelanja.length);

```

Outputnya :

```
Setelah ditambah:
[
  'Beras',  'Gula',
  'Minyak', 'Telur',
  'Garam',  'Susu',
  'Kopi'
]

Item yang dihapus: Beras

'Susu' ada di dalam daftar belanja

Jumlah item akhir: 6
```

Latihan 4

```javascript
// 1. Deklarasi array of objects
const produk = [
  { nama: "Laptop", harga: 8500000, stok: 5 },
  { nama: "Mouse", harga: 150000, stok: 0 },
  { nama: "Keyboard", harga: 450000, stok: 12 },
  { nama: "Monitor", harga: 3200000, stok: 0 },
  { nama: "Headset", harga: 350000, stok: 8 },
];

// 2. Filter produk yang tersedia (stok > 0)
const produkTersedia = produk.filter((item) => item.stok > 0);

console.log("Produk tersedia:");
console.log(produkTersedia);

// 3. Ambil hanya nama produk (map)
const namaProduk = produkTersedia.map((item) => item.nama);
console.log("\nNama produk yang tersedia:");
console.log(namaProduk);

// 4. Hitung total nilai inventaris (reduce)
const totalInventaris = produkTersedia.reduce((total, item) => {
  return total + item.harga * item.stok;
}, 0);

console.log("\nTotal nilai inventaris: Rp" + totalInventaris);

// 5. Tampilkan semua produk (forEach)
console.log("\nDaftar semua produk:");
produk.forEach((item) => {
  const status = item.stok > 0 ? "[TERSEDIA]" : "[HABIS]";

  if (item.stok > 0) {
    console.log(
      `${status} ${item.nama} - Rp${item.harga.toLocaleString()} (${item.stok} unit)`,
    );
  } else {
    console.log(`${status} ${item.nama} - Rp${item.harga.toLocaleString()}`);
  }
});
```

Outputnya :

```
[
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Headset', harga: 350000, stok: 8 }
]

Nama produk yang tersedia:
[ 'Laptop', 'Keyboard', 'Headset' ]

Total nilai inventaris: Rp50700000

Daftar semua produk:
[TERSEDIA] Laptop - Rp8,500,000 (5 unit)
[HABIS] Mouse - Rp150,000
[TERSEDIA] Keyboard - Rp450,000 (12 unit)
[HABIS] Monitor - Rp3,200,000
[TERSEDIA] Headset - Rp350,000 (8 unit)
```

Langkah 11 || Selanjutnya kita membuat folder tugas dengan cara masukan perintah `mkdir tugas` lalu untuk membuat file nya masukan perintah `touch tugas-1.js tugas-2.js`

JS tugas-1.js

```javascript
// 1. Data mahasiswa (minimal 6)
const dataMahasiswa = [
  { nama: "Andi", nilai: 85 },
  { nama: "Budi", nilai: 70 },
  { nama: "Citra", nilai: 90 },
  { nama: "Dina", nilai: 60 },
  { nama: "Eka", nilai: 75 },
  { nama: "Fajar", nilai: 95 },
];

// 2. Function hitung statistik (pakai reduce)
function hitungStatistik(arrMahasiswa) {
  const hasil = arrMahasiswa.reduce(
    (acc, mhs, index) => {
      acc.total += 1;
      acc.totalNilai += mhs.nilai;
      acc.tertinggi = Math.max(acc.tertinggi, mhs.nilai);
      acc.terendah = Math.min(acc.terendah, mhs.nilai);

      if (index === arrMahasiswa.length - 1) {
        acc.rataRata = acc.totalNilai / acc.total;
      }

      return acc;
    },
    {
      total: 0,
      totalNilai: 0,
      rataRata: 0,
      tertinggi: -Infinity,
      terendah: Infinity,
    },
  );

  return {
    total: hasil.total,
    rataRata: hasil.rataRata,
    tertinggi: hasil.tertinggi,
    terendah: hasil.terendah,
  };
}

// 3. Filter lulus
function filterLulus(arrMahasiswa, batasLulus) {
  return arrMahasiswa.filter((mhs) => mhs.nilai >= batasLulus);
}

// 4. Tambah grade
function tambahGrade(arrMahasiswa) {
  return arrMahasiswa.map((mhs) => {
    let grade;

    if (mhs.nilai >= 85) grade = "A";
    else if (mhs.nilai >= 75) grade = "B";
    else if (mhs.nilai >= 65) grade = "C";
    else if (mhs.nilai >= 50) grade = "D";
    else grade = "E";

    return { ...mhs, grade };
  });
}
```

```javascript

// 5. Menjalankan semua function
const statistik = hitungStatistik(dataMahasiswa);
const mahasiswaLulus = filterLulus(dataMahasiswa, 75);
const mahasiswaDenganGrade = tambahGrade(dataMahasiswa);

// 6. Output rapi
console.log("=== STATISTIK MAHASISWA ===");
console.log(`Total: ${statistik.total}`);
console.log(`Rata-rata: ${statistik.rataRata.toFixed(2)}`);
console.log(`Nilai Tertinggi: ${statistik.tertinggi}`);
console.log(`Nilai Terendah: ${statistik.terendah}`);

console.log("\n=== MAHASISWA LULUS ===");
mahasiswaLulus.forEach((mhs, i) => {
  console.log(`${i + 1}. ${mhs.nama} - ${mhs.nilai}`);
});

console.log("\n=== DATA DENGAN GRADE ===");
mahasiswaDenganGrade.forEach((mhs, i) => {
  console.log(`${i + 1}. ${mhs.nama} - ${mhs.nilai} (Grade ${mhs.grade})`);
});

```

Outputnya :

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010053_Algoritma_Dan_Struktur_Data/praktikum-3/tugas (main)
$ node tugas-1.js
=== STATISTIK MAHASISWA ===
Total: 6
Rata-rata: 79.17
Nilai Tertinggi: 95
Nilai Terendah: 60

=== MAHASISWA LULUS ===
1. Andi - 85
2. Citra - 90
3. Eka - 75
4. Fajar - 95

=== DATA DENGAN GRADE ===
1. Andi - 85 (Grade A)
2. Budi - 70 (Grade C)
3. Citra - 90 (Grade A)
4. Dina - 60 (Grade D)
5. Eka - 75 (Grade B)
6. Fajar - 95 (Grade A)
```

JS tugas-2.js

```javascript
function pangkat(basis, eksponen) {
  // base case
  if (eksponen === 0) {
    return 1;
  }

  // recursive case
  return basis * pangkat(basis, eksponen - 1);
}

// uji coba pangkat
console.log("=== UJI PANGKAT ===");
console.log(pangkat(2, 3)); // 8
console.log(pangkat(5, 2)); // 25
console.log(pangkat(3, 4)); // 81

// ========================================
// 2. Function Rekursif Balik String
// ========================================
function balikString(str) {
  // base case
  if (str.length <= 1) {
    return str;
  }

  // recursive case
  return str[str.length - 1] + balikString(str.slice(0, str.length - 1));
}

// uji coba balik string
console.log("\n=== UJI BALIK STRING ===");
console.log(balikString("halo")); // olah
console.log(balikString("ihsan")); // nasih

// ========================================
// 3. Function Cek Palindrom
// ========================================
function cekPalindrom(str) {
  let hasilBalik = balikString(str);
  return str === hasilBalik;
}

// uji coba palindrom
console.log("\n=== UJI PALINDROM ===");
console.log("katak:", cekPalindrom("katak")); // true
console.log("civic:", cekPalindrom("civic")); // true
console.log("level:", cekPalindrom("level")); // true
console.log("halo :", cekPalindrom("halo")); // false
```

Outputnya :

```
Administrator@DESKTOP-CJ2TM4F MINGW64 ~/2025573010053_Algoritma_Dan_Struktur_Data/praktikum-3/tugas (main)
$ node tugas-2.js
=== UJI PANGKAT ===
8
25
81

=== UJI BALIK STRING ===
olah
nashi

=== UJI PALINDROM ===
katak: true
civic: true
level: true
halo : false
```

Langkah 12 || Push project dari lokal ke GitHub

BAB IV

HASIL DAN KESIMPULAN

4.1 Hasil Praktikum

Pada praktikum Modul 3 ini, telah dibuat beberapa fungsi (functions) dan pengolahan data menggunakan array dalam bahasa pemrograman JavaScript. Fungsi-fungsi yang dibuat meliputi operasi dasar seperti penjumlahan, pengurangan, perkalian, dan pembagian, serta fungsi yang memanfaatkan array untuk mengolah sekumpulan data.

Selain itu, dilakukan juga pembuatan fungsi dengan konsep rekursif dan penggunaan method bawaan array seperti map(), filter(), dan reduce() untuk memproses data secara lebih efisien.

Hasil dari praktikum menunjukkan bahwa:

- Fungsi dapat digunakan untuk mengelompokkan logika program agar lebih terstruktur.
- Array memudahkan dalam menyimpan dan mengelola banyak data.
- Kombinasi function dan array membuat program menjadi lebih fleksibel dan mudah dikembangkan.

4.2 Pembahasan

4.2.1 Function (Fungsi)

Function merupakan blok kode yang dirancang untuk melakukan tugas tertentu dan dapat dipanggil kembali saat dibutuhkan. Penggunaan function sangat membantu dalam menghindari penulisan kode yang berulang.

Dalam praktikum ini, function digunakan untuk:

- Operasi matematika dasar
- Validasi input (seperti pembagian dengan nol)
- Pengolahan data

Dengan adanya function, program menjadi lebih modular dan mudah dipahami.

4.2.2 Array

Array adalah struktur data yang digunakan untuk menyimpan banyak nilai dalam satu variabel. Pada praktikum ini, array digunakan untuk menyimpan data mahasiswa beserta nilainya.

Pengolahan array dilakukan dengan beberapa metode:

- map() untuk memodifikasi data
- filter() untuk menyaring data
- reduce() untuk menghitung nilai total atau rata-rata

Penggunaan array sangat efektif dalam mengelola data dalam jumlah banyak.

4.2.3 Penggunaan Reduce untuk Statistik

Method reduce() digunakan untuk menghitung nilai statistik seperti total, rata-rata, nilai tertinggi, dan terendah dari data mahasiswa.

Dengan reduce(), proses perhitungan dapat dilakukan dalam satu iterasi sehingga lebih efisien dibandingkan metode konvensional.

4.2.4 Function Rekursif

Function rekursif adalah fungsi yang memanggil dirinya sendiri. Pada praktikum ini digunakan untuk menghitung perpangkatan dan membalik string.

Rekursi mempermudah penyelesaian masalah yang bersifat berulang, namun harus memiliki kondisi berhenti (base case) agar tidak terjadi infinite loop.

4.2.5 Analisis Hasil

Dari hasil praktikum, dapat disimpulkan bahwa:

- Method modern JavaScript sangat membantu dalam efisiensi coding
- Array mempermudah pengolahan data kompleks

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan pada Modul 3 tentang Functions dan Array, dapat disimpulkan bahwa:

1. Function sangat penting dalam pemrograman untuk membuat kode lebih terstruktur dan reusable.
2. Array memudahkan penyimpanan dan pengolahan data dalam jumlah banyak.
3. Method array seperti map(), filter(), dan reduce() sangat membantu dalam manipulasi data.
4. Function rekursif dapat digunakan untuk menyelesaikan permasalahan tertentu secara efisien.
5. Kombinasi function dan array membuat program lebih efektif, efisien, dan mudah dikembangkan.

5.2 Saran

Adapun saran yang dapat diberikan setelah melakukan praktikum ini adalah:

1. Mahasiswa diharapkan lebih sering berlatih menggunakan function dan array dalam berbagai kasus.
2. Memahami konsep dasar terlebih dahulu sebelum menggunakan method yang lebih kompleks.
3. Mencoba mengembangkan program dengan studi kasus yang lebih nyata agar pemahaman semakin mendalam.
4. Menggunakan kode yang rapi dan terstruktur agar mudah dibaca dan dipelihara.